

조선소 선박 블록 배치 최적화

퍼즐 같은 이 문제를, 코드는 어떻게 푸는가

이 문서는

프로그래밍이나 수학 전공 지식이 전혀 없어도 읽을 수 있도록 만들었습니다. 먼저 "이 문제가 도대체 뭔지"를 일상 비유로 설명하고, 그다음에 "컴퓨터가 어떻게 푸는지"와 "우리가 짠 코드 파일 6개가 각각 무슨 일을 하는지"를 차근차근 풀어 갑니다.

기준 문서

OGC 2026: The Grand Shipyard Puzzle / LG CNS 배치 최적화 기획안

다루는 코드

geometry · evaluator · candidates · greedy · alns · myalgorithm

난이도

비전공자용 (비유와 그림 중심)

1. 큰 그림부터: 이게 무슨 문제예요?

거대한 배 한 척은 레고처럼 여러 개의 큰 덩어리(블록)를 따로 만든 뒤 나중에 합쳐서 완성합니다. 이 블록들은 아무 데서나 만들 수 없고, 정해진 작업장(베이, bay) 안에서 만들어집니다. 문제는 작업장이 넓지 않고, 만들어야 할 블록은 많다는 것입니다.

한 문장 요약

"어떤 블록을 / 어느 작업장에 / 어떤 방향으로 돌려서 / 어디에 놓고 / 언제 넣었다가 / 언제 뺄지" — 이 여섯 가지를 한꺼번에 똑똑하게 정하는 퍼즐입니다.

익숙한 것에 빗대면 이렇습니다.

- 테트리스처럼 — 모양이 제각각인 조각을 빈틈없이 끼워 넣어야 합니다. 다만 조각이 네모가 아니라 울퉁불퉁합니다.
- 주차장처럼 — 차(블록)마다 들어오는 날과 빼야 하는 날이 다르고, 자리가 겹치면 안 됩니다.
- 호텔 예약처럼 — 같은 방(작업장)을 시간을 나눠 여러 손님(블록)이 쓰는데, 동시에 머무는 손님끼리는 자리가 부딪치면 안 됩니다.

Bay (작업장) — 가로 W × 세로 H 의 빈 공간



그림 1. 하나의 작업장(베이) 안에 블록 두 개를 서로 겹치지 않게 나란히 배치한 모습

2. 등장인물(용어)과 친해지기

이 문제에는 몇 가지 단어가 반복해서 나옵니다. 딱 이만큼만 알면 됩니다.

베이 (bay)	블록을 만드는 직사각형 작업장. 가로 W, 세로 H 크기가 정해져 있습니다.
블록 (block)	배의 한 덩어리. 네모가 아니라 울퉁불퉁한 3D 입체이고, 여러 "층(layer)"이 쌓여 있습니다.
층 (layer)	블록을 옆에서 봤을 때 보이는 단(段). 1층은 넓고 위로 갈수록 좁아지는 식입니다.
방향 (orientation)	블록을 회전시켜 놓는 방법. 같은 블록도 여러 방향으로 돌려 끼울 수 있습니다.
투입가능일 (release)	이 블록 작업을 시작할 수 있는 가장 빠른 날.
처리시간 (processing)	블록을 만드는 데 걸리는 날 수.
납기일 (due)	이 날까지는 끝내고 싶은 목표일. 넘기면 "지연"입니다.
ENTRY / EXIT	크레인으로 블록을 작업장에 넣는 일 / 빼는 일.

3. 무엇을 "결정"해야 하나요? (블록 하나당 6가지)

블록 하나를 처리하려면 아래 여섯 가지를 정해야 합니다. 어렵게 들리지만 일상 언어로는 단순합니다.

1. 어느 작업장?	여러 베이 중 한 곳을 고른다.	"3번 주차장에 댄다"
2. 어느 방향?	블록을 어떻게 돌려 놓을지.	"세로로 세운다 vs 눕힌다"
3. 어느 자리?	베이 안의 정확한 (가로, 세로) 위치.	"왼쪽 구석부터"
4. 언제 넣지? (ENTRY)	며칠에 투입할지.	"5일째 아침에 넣는다"
5. 언제 빼지? (EXIT)	며칠에 반출할지.	"8일째 뺀다"

왜 어렵냐면

이 여섯 가지가 서로 얽혀 있기 때문입니다. 한 블록을 좋은 자리에 일찍 넣으면, 바로 그 자리·그 시간에 다른 블록이 못 들어갑니다. 그래서 한 블록만 잘 두는 게 아니라 전체를 동시에 맞춰야 하는 퍼즐이 됩니다.

4. 무엇을 "잘"하면 좋은 점수인가요? (목표 3가지)

정답이 하나가 아니라, "더 좋은 배치"와 "덜 좋은 배치"가 있습니다. 좋고 나쁨은 아래 세 가지 별점의 합으로 매깁니다. 별점이 작을수록 좋은 답입니다.

목표 1 — 납기 지연 (Z_1)

약속한 날(납기일)보다 늦게 끝낸 날 수를 모두 더합니다. 택배가 늦으면 늦은 만큼 불만이 쌓이는 것과 같습니다.

시간 → 납기일(due)을 지나서 EXIT하면 그만큼 "지연"으로 점수가 깎입니다.

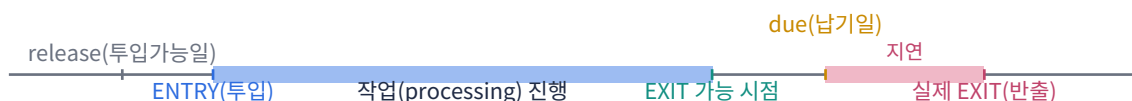


그림 2. 한 블록의 일생: 투입가능일 이후에 넣고(ENTRY), 작업을 마치고 빼는데(EXIT), 납기일을 넘기면 그만큼이 지연 별점

목표 2 — 작업장 간 일 몰림 (Z_2)

한 작업장에만 일이 몰리고 다른 작업장은 놀면 비효율적입니다. 작업장마다 떠안은 일의 양을 비교해서, 가장 바쁜 곳과 가장 한가한 곳의 차이를 별점으로 봅니다. 단, 큰 작업장은 여유가 있으니 같은 일이라도 덜 붐비는 것으로 쳐 줍니다.

목표 3 — 선호 무시 별점 (Z_3)

블록마다 "이왕이면 이 작업장이 좋다"는 선호 점수가 있습니다(설비 궁합 등). 가장 좋아하는 곳에 못 넣고 덜 좋은 곳에 넣으면, 그 아쉬운 만큼이 별점이 됩니다.

총점 공식

최종 점수 = $w_1 \times (\text{납기지연}) + w_2 \times (\text{일 몰림}) + w_3 \times (\text{선호 무시})$. w_1, w_2, w_3 는 "어느 목표를 얼마나 중요하게 볼지" 정하는 가중치이고, 문제마다 주어집니다. 우리 목표는 이 총점을 최대한 작게 만드는 것입니다.

5. 절대 어기면 안 되는 규칙 (제약 3가지)

점수를 잘 받기 전에, 먼저 "말이 되는" 배치여야 합니다. 아래 규칙을 하나라도 어기면 그 답은 무효(0점이 아니라 감점)입니다.

규칙 1 — 작업장 밖으로 빠져나가면 안 됨

블록의 모든 층이 작업장 사각형 안에 쏙 들어와야 합니다. 벽에 딱 붙는 것은 괜찮지만 한 귀퉁이라도 밖으로 나가면 안 됩니다.

규칙 2 — 같은 시간, 같은 층끼리 겹치면 안 됨

같은 작업장에 같은 기간 동안 머무는 두 블록은 서로 부딪치면 안 됩니다. 재미있는 점은 "같은 높이의 층끼리만" 따진다는 것입니다. 위에서 보면 겹쳐 보여도, 한쪽은 1층만 있고 다른 쪽은 그 위 2층이 튀어나온 모양이라면 실제로는 안 부딪칠 수 있습니다. (블록이 계단처럼 생겼기 때문입니다.)

규칙 3 — 크레인 길이 막히면 안 됨

블록은 크레인으로 위에서 아래로 수직으로 내리고 올립니다. 그래서 넣거나 뺄 때, 그 블록의 어떤 층이든 같은 높이거나 더 높은 옆 블록과 부딪치면 길이 막혀 작업이 불가능합니다. 낮게 깔린 블록 위로는 크레인이 지나갈 수 있지만, 키 큰 블록은 옆 블록의 윗부분에 걸릴 수 있다는 뜻입니다.

우리 코드의 약속

규칙 3을 안전하게 지키려고, 우리 코드는 "두 블록이 같은 작업장에 같이 있을 거면, 넣을 때도 뺄 때도(순서가 어떻든) 크레인이 안 막히는 경우만 허용"하는 보수적인 안전 규칙을 씁니다. 살짝 손해를 보더라도 무효답이 나오지 않게 한 것입니다.

6. 컴퓨터는 이걸 어떤 순서로 푸나요?

한 번에 완벽한 답을 찾기는 너무 어렵습니다. 그래서 요리처럼 단계를 나눠 진행합니다. 아래가 전체 흐름이고, 각 단계가 코드 파일 하나에 대응합니다.

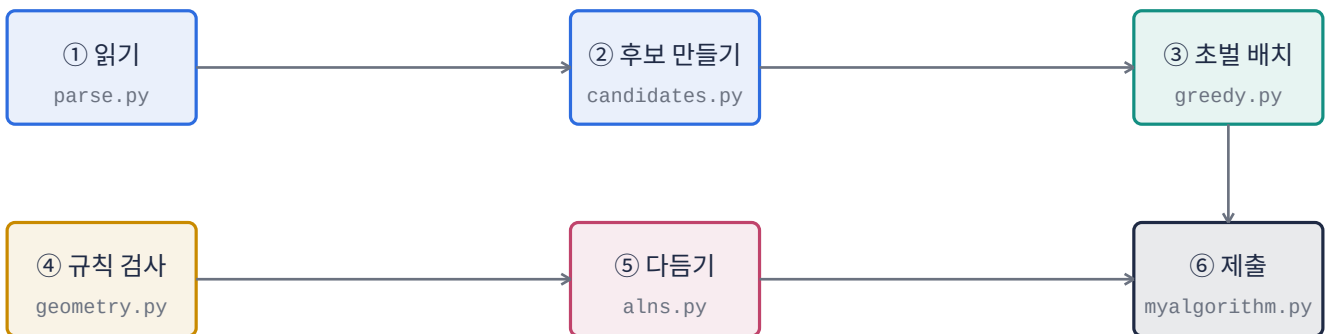


그림 3. 데이터를 읽어 → 놓을 후보 자리를 만들고 → 초벌로 배치한 뒤 → 규칙을 검사하고 → 더 좋게 다듬어 → 제출하는 흐름

7. 우리가 만든 코드 파일 6개 (하나씩 소개)

각 파일을 "부역의 도구"에 빗대 설명합니다. 코드를 못 읽어도 무슨 일을 하는지 알 수 있게 했습니다.

geometry.py — 규칙 심판

비유 블록이 벽을 넘었는지, 서로 부딪쳤는지, 크레인이 막혔는지 판정하는 **심판**.

하는 일 블록의 울퉁불퉁한 모양을 좌표로 받아, 겹침·빠져나감·크레인 간섭을 계산합니다. 또 날짜순으로 넣고 빼는 과정을 처음부터 끝까지 훑내 내며 "이 답이 규칙을 다 지켰나"를 검사합니다.

evaluator.py — 점수 계산기

비유 시험 채점관처럼 한 답안의 **총점(별점)**을 매깁니다.

하는 일 납기 지연(Z1), 일 몰림(Z2), 선호 무시(Z3)를 각각 계산하고 가중치를 곱해 최종 점수를 냅니다. 문제정의서의 공식을 그대로 옮겼고, 손계산 예제로 정확함을 확인했습니다.

candidates.py — 자리 추천기

비유 "여기 놓을 만해요" 하고 **후보 자리들을 골라 주는** 안내원.

하는 일 가능한 위치가 사실상 무한히 많기 때문에, 작업장 구석·이미 놓인 블록의 모서리 같은 쓸 만한 지점만 추려 줍니다. 덕분에 컴퓨터가 헛수고를 덜 합니다.

greedy.py — 초벌 배치공

비유 일단 빠르게 "그럭저럭 괜찮은" 첫 답을 만드는 **밑그림 화가**.

하는 일 급한 블록(납기 임박·큰 블록)부터 순서를 정해 자리에 넣습니다. 두 가지 방식이 있는데, 하나는 안전한 "직렬", 하나는 더 알찬 "패킹"입니다(다음 장 참고).

alns.py — 다듬기 장인

비유 완성된 그림에서 어색한 부분만 **지웠다 다시 그리는** 리터치 작가.

하는 일 지연이 큰 블록 몇 개를 빼낸 뒤 더 좋은 자리로 다시 넣기를 반복합니다. 가끔은 잠깐 나빠지는 선택도 허용해 더 좋은 답으로 빠져나갑니다(담금질 기법).

myalgorithm.py — 총괄 지휘자

비유 대회 서버가 호출하는 정문(entry point).

하는 일 안전한 답을 먼저 확보해 두고, 시간 안에서 패킹·다듬기로 개선하되 반드시 규칙 검사를 통과한 답만 채택합니다. 그래서 어떤 경우에도 무효답을 내지 않습니다.

8. 초벌 배치의 두 가지 방식

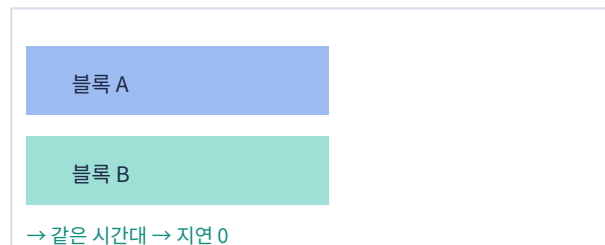
greedy.py의 핵심 아이디어입니다. 같은 두 블록을 두는데 결과가 크게 달라집니다.

직렬(serial): 한 번에 하나씩



시간 →

패킹(packed): 나란히 동시에



시간 →

그림 4. (왼쪽) 직렬: 한 작업장을 한 번에 한 블록씩 → 안전하지만 뒤 블록이 늦어짐. (오른쪽) 패킹: 부딪치지 않으면 같은 시간에 나란히 → 지연을 줄임

실제로 돌려 본 결과

작업장 하나에 블록 두 개를 넣는 작은 예제에서 — 직렬 방식은 뒤 블록이 늦어 **벌점 100**이 나왔고, **패킹** 방식은 둘을 나란히 놓아 **벌점 0**(지연 없음)을 달성했습니다. 두 답 모두 심판(geometry)의 규칙 검사를 통과했고, 총괄 지휘자(myalgorithm)는 더 좋은 패킹 답을 골랐습니다.

9. 우리가 풀어낸 작은 미스터리: "제곱근($\sqrt{\quad}$)이 있나 없나"

문제정의서의 둘째 목표(Z2) 공식에 큰 기호가 그려져 있어, 제곱근($\sqrt{\quad}$)인지 단순 절댓값인지 헷갈렸습니다. 이 한 글자 차이가 점수 계산을 통째로 바꿉니다.

그래서 정의서에 실린 예시 화면의 숫자로 **거꾸로 계산**했습니다. 화면에 적힌 최종 점수와 세부 값을 가중치로 맞춰 보니 — 제곱근 없이 계산했을 때 딱 맞아떨어졌습니다.

검산

$56 \times 26667 + 10 \times 625.9075 + 300 \times 427 = 1,627,711.075 \rightarrow$ 화면의 점수와 정확히 일치. 따라서 Z2에는 **제곱근이 없다**고 확정하고 코드 기본값을 그렇게 맞췄습니다. (그래도 제출 전에는 대회가 준 검사 도구로 한 번 더 대조하길 권합니다.)

10. 더 잘하려면 다음에 무엇을 하면 되나요?

- 다듬기 강화 — 지금은 다듬을 때 전체를 다시 그리는데, 뺀 블록만 부분적으로 다시 넣게 바꾸면 훨씬 빨라지고 좋아집니다.
- 수학 최적화 추가 — 짧은 기간만 떼어내 전문 최적화 엔진(Gurobi 등)으로 정밀하게 다시 푸는 단계를 붙입니다.
- 대시보드 — 날짜별 작업장 그림, 블록별 일정표(간트차트), 점수 분해를 화면으로 보여 주면 현업 설명이 쉬워집니다.

마무리 한마디

이 문제는 "예측"이 아니라 "현장의 뻘뻘한 제약 속에서 가장 합리적인 결정을 찾는" 운영 최적화입니다. 완벽한 3D를 처음부터 노리기보다, 네모 블록 → 울퉁불퉁 블록 → 층 쌓인 블록 → 크레인 → 다듬기 순으로 **한 계단씩** 키워 가는 것이 가장 안정적인 길입니다.

부록. 한 장 용어 사전

최적화	여러 가능한 답 중에서 "가장 좋은(별점이 가장 작은)" 답을 찾는 일.
제약 (constraint)	반드시 지켜야 하는 규칙. 어기면 그 답은 무효.
목적함수 (objective)	답이 얼마나 좋은지 숫자로 나타낸 점수. 작을수록 좋음.
feasible (실행가능)	모든 규칙을 지켜서 "말이 되는" 답.
greedy (탐욕법)	매 순간 당장 좋아 보이는 선택을 빠르게 이어 붙여 답을 만드는 방법.
휴리스틱	완벽함을 보장하진 않지만 현실적으로 좋은 답을 빨리 찾는 요령.
ALNS	답의 일부를 부수고 더 좋게 다시 짓기를 반복하는 개선 기법.
간트차트	누가 언제부터 언제까지 일하는지 막대로 보여 주는 일정표.

이 안내서는 OGC 2026 문제정의서와 LG CNS 기획안을 바탕으로, 함께 작성한 코드(geometry·evaluator·candidates·greedy·alns·myalgorithm)를 비전공자용으로 풀어 쓴 것입니다.